

BIG DATA ANALYTICS SYSTEM

Healthcare Patient Data Analysis Using HDFS and Apache Spark

Course:

Big Data Programming (Coursework)

Module Leader:

Ms. Duleesha Manohari

Group Members:

W G M M Ajmal Mansoor (26775)

K.B.A.Pasandul (30452)

Chandrasinghe S S (29793)

ABS Mahanama (27818)

Pathirana RV (30486)

GitHub Repo: <https://github.com/M-Ajmal/healthcare-data-analysis-pyspark.git>

Table of content

1.	Introduction.....	4
1.1	Overview of Big Data Concepts	4
1.2	Project Overview	5
2.	Problem Definition.....	5
2.1	Problem Scenario	5
2.2	Why This Is a Big Data Problem	6
2.2.1	Data Volume	6
2.2.2	Data Velocity	6
2.2.3	Data Variety	6
3.	Dataset Description.....	7
3.1	Source and Generation.....	7
3.2	Structure and Attributes	7
3.3	Preprocessing Steps	8
3.3.1	Data Cleaning.....	8
3.3.2	Feature Engineering.....	9
4.	System Architecture.....	10
4.1	Architectural Overview.....	10
4.2	Storage Layer: HDFS.....	10
4.3	Processing Layer: Apache Spark	11
4.4	Environment: Ubuntu and Jupyter	11
4.5	Technology Justification.....	11
5.	Methodology.....	12
5.1	Overview of the Three-Phase Pipeline	12
5.2	Phase 1: Data Ingestion and Preparation	12
5.3	Phase 2: Distributed Processing with Apache Spark	12
5.4	Phase 3: Visualisation	12
6.	Implementation	13
7.	Results and Evaluation.....	14
7.1	Disease Prevalence Analysis.....	14
7.2	Test Results Distribution.....	15
7.3	Demographic Analysis.....	16
7.4	Anomaly Detection Results	17
8.	Discussion	19

8.1	Challenges Encountered.....	19
8.2	Limitations	19
8.3	Performance and Scalability	20
9.	Conclusion	21
10.	Workload Distribution	21
11.	References	22

1. Introduction

1.1 Overview of Big Data Concepts

The advent of digital technologies has resulted in the generation of more data in many industries, including health care. This vast and diverse data, known as big data, encompasses patient data, health records, diagnostic tests, payment information and hospital administrative data. These data sets are characterised by their volume, velocity, variety, veracity and value, making them challenging to manage using conventional data processing methods.

This data requires sophisticated tools and technologies, including distributed and parallel processing platforms, to manage and analyse it. Apache Spark is a popular big data processing platform that offers efficient and scalable data analytics capabilities through in-memory processing and distributed computing. This is particularly crucial in the healthcare industry, where the ability to gain insights from data in a timely manner can have a direct impact on patient care and operational efficiency.

This project introduces a scalable framework for healthcare data analytics that performs data processing and analysis on patient data using Apache Spark. The system carries out several data manipulation tasks such as aggregation, filtering, sorting, and joining, in a data pipeline. It aims to provide insights into important parameters such as disease prevalence, patient age and gender, test result trends, and hospital-related anomalies.

Specifically, the system detects key patterns like abnormal test results, extended hospital stays and high-cost cases. These patterns help in identifying health risks, disease patterns within a population, and managing hospital resources effectively. The system uses big data technologies to provide data-driven insights for healthcare practitioners and administrators, leading to better patient treatment and hospital efficiency.

1.2 Project Overview

This project demonstrates a big data analytics pipeline, using a pseudo-random sample of 55,500 patient admissions. It showcases distributed data storage, analytics, and anomaly detection in a real data engineering scenario. The system is based on data engineering with pandas for data cleaning and transformation, distributed analytics with Apache PySpark on a local multicore cluster, and visualisation with matplotlib and seaborn. The analytics discover common diagnoses, examine patient demographics, and detect anomalies in patient stays, costs, and blood test results, all using Python on Ubuntu Linux.

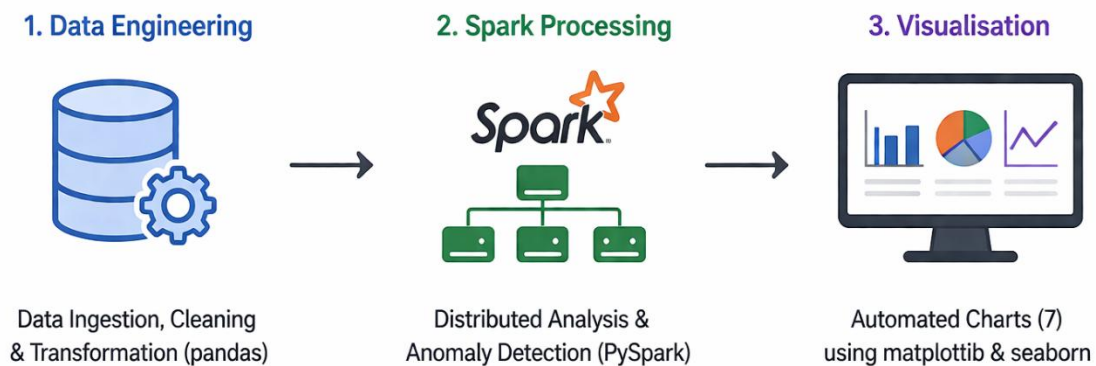


Figure 1: Big Data Analytics Pipeline Phases

2. Problem Definition

2.1 Problem Scenario

Hospitals produce enormous amounts of patient data as part of routine activities including patient admissions, diagnoses, treatments and discharges. These datasets contain a variety of information such as patient demographics, clinical observations, diagnostic outcomes, and billing records. When collected from multiple hospitals and over long time spans, these data sets are extremely valuable in supporting epidemiological research, enhancing clinical decision-making, resource allocation and financial fraud detection.

However, the size and complexity of healthcare data present challenges. Conventional data processing tools and single-threaded methods are not efficient enough to process large volumes of

data with different structures. This makes it challenging to draw important conclusions, including trends in disease occurrence, risk factors in populations and anomalies in patient stays or claims.

As such, the primary challenge of this project is to design and implement a scalable and efficient data processing system that ingests, cleans, transforms and analyzes big healthcare data. It should also automatically identify relevant clinical and financial anomalies and be reproducible and efficient. This requires the development of a distributed data processing pipeline that can efficiently and reliably process big data.

2.2 Why This Is a Big Data Problem

2.2.1 Data Volume

There are 55,500 records of patients with 16 features. Following transformation, there are 8 additional attributes, bringing the total to 19. While small, the system is scalable and uses technologies such as HDFS, Parquet and Spark which can handle much larger data.

2.2.2 Data Velocity

The data source used to date is batch-oriented, but the system can handle high-velocity data. Through Spark Structured Streaming, streaming data from the hospital (admissions, lab results, billing) can be processed with limited modifications as the pipeline is modular.

2.2.3 Data Variety

The data set contains different types of data - categorical, numerical, temporal. The pipeline processes these with suitable transformations such as encoding, date manipulations and normalisation, demonstrating the variety component of big data.

3. Dataset Description

3.1 Source and Generation

This project uses a synthetic health care patient admission dataset of 55,500 rows, stored as a CSV file on HDFS. This dataset mimics real-world hospital data, containing patient demographics, diagnoses, admission information, billing and lab results. Being synthetic data, it guarantees no patient confidentiality is breached, so it can be used for educational and research purposes

3.2 Structure and Attributes

The entire attribute schema of the processed dataset is shown in Figure 2, which includes both the original variables and the newly created features produced by the data engineering workflow.

	Attribute	Data Type	Description
	Name	str	Patient name
	Age	Integer	Patient age in years (0–120)
	Gender	str	Male / Female
	Blood Type	str	A+, A-, B+, B-, AB+, AB-, O+, O-
	Medical Condition	str	Primary diagnosis
	Date of Admission	str	Admission date
	Doctor	str	Doctor name
	Hospital	str	Hospital name
	Insurance Provider	str	Insurance company
	Billing Amount	Float	Total bill in USD
	Room Number	Integer	Room number
	Admission Type	str	Elective / Urgent / Emergency
	Discharge Date	str	Discharge date
	Medication	str	Prescribed medication
	Test Results	str	Normal / Inconclusive / Abnormal
	Length of Stay	Integer	Days stayed in hospital

Figure 2 Dataset Attribute Schema

There were sixteen attributes in the raw dataset at first. The final processed dataset includes 19 analytical attributes after deleting personally identifying information (PII) elements, such as patient name, attending physician, and room number, and adding eight more derived features.

3.3 Preprocessing Steps

3.3.1 Data Cleaning

The data In the data cleaning step (`clean_data.py`), there are some key steps. First, the column names are converted to snake_case to make them easier to use in code and to comply with the Spark format. Then, unnecessary and potentially sensitive columns are dropped to hide patient privacy and reduce the number of columns. Next, any spaces in all text columns are stripped, and all categorical columns are standardized into Title Case (for example, “male” → “Male”). The blood_type column is also capitalised. Then, all of the date columns are converted from text to datetime. When date conversion is not possible, the date is marked as NaT (Not a Time). We also correct the columns with numbers:

- billing_amount is converted to a float
- age is converted to an integer (no trailing 0s - 45 not 45.0)

The length_of_stay is then calculated based on admission and discharge dates (instead of the original value) to reduce errors. And finally, we remove invalid cases.

This includes cases where:

- age is not in the range 0-120
- length of stay is negative
- discharge date is earlier than admission date

3.3.2 Feature Engineering

The feature engineering, in `transform_data.py`, creates new informative columns.

We divide age into four groups:

- Child (0–12)
- Teenager (13–17)
- Adult (18–60)
- Senior (61–120)

The length of stay is also grouped into:

- Short (0–7 days)
- Medium (8–30 days)
- Long (more than 30 days)

The amount billed is broken down into three categories:

- Low (less than \$25,000)
- Medium (\$25,000–\$60,000)
- High (more than \$60,000)

Also, some temporal features such as `year`, `month` and `day of the week` are extracted from `date of admission` for temporal analysis. Finally, some features such as `test_results` and `admission_type` are converted to numeric (ordinal) values to help with machine learning.

4. System Architecture

4.1 Architectural Overview

Our system is a three-stage, modular pipeline that employs a traditional ETL approach to distributed data processing. The system has three layers: storage layer, processing layer and presentation layer. The data is processed in a pipelined fashion through each layer with intermediate results saved at designated points to allow re-processing of individual phases without needing to re-process previous steps. The system architecture is shown in Figure 1 below.

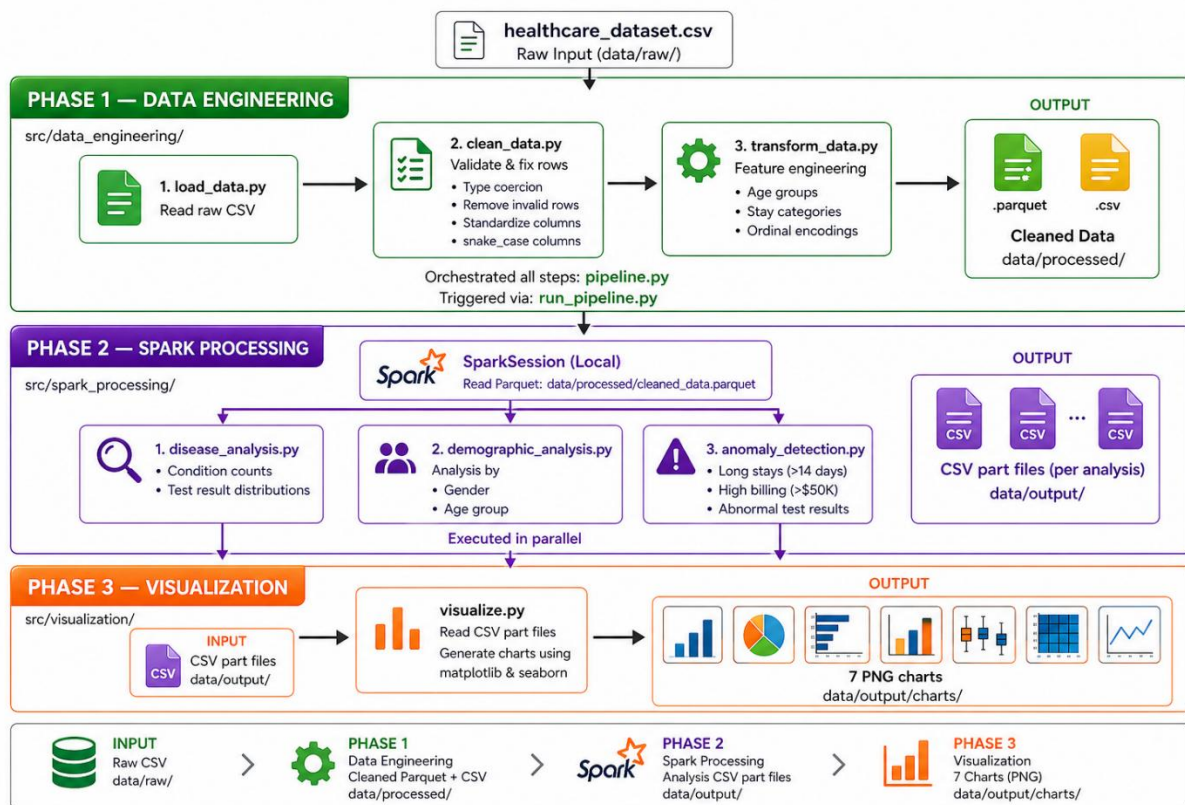


Figure 3: System Architecture : Three-Phase Big Data Pipeline

4.2 Storage Layer: HDFS

The raw CSV data is first copied to the HDFS. HDFS is a distributed file system that allows the storage of very large files across a cluster of commodity servers, offering high aggregate bandwidth across the cluster, automatic replication (the default is three copies) for redundancy and fault tolerance. For this project, HDFS is the source of truth for the data, and the raw data is

accessible to any node in the cluster and is resilient to individual node failures. The data is copied from HDFS to the local filesystem for processing by the data engineering layer (pandas-based data processing pipeline) and the cleaned (Parquet) file is the input to the Spark processing layer. In a production system, the raw and processed files would not be exported, and Spark would ingest from hdfs:// URIs

4.3 Processing Layer: Apache Spark

The processing is done with Apache Spark on an Ubuntu system in local multi-core mode, making use of all available CPU cores. The preprocessed data is read from a Parquet file with a local file path (file://) to bypass HDFS problems. It is loaded as a Spark DataFrame, which is distributed and evaluated only when required (lazy evaluation). We use functions such as groupBy, filter and agg for data analysis, which are automatically optimized by Spark. All the results are stored as CSV files in multiple parts and then merged for visualisation.

4.4 Environment: Ubuntu and Jupyter

Development occurs on Ubuntu, which can run Java (for Spark and HDFS) and Python packages such as pandas and matplotlib. The Jupyter Notebook is utilized for data exploration and validating findings progressively. Data exploration plays an essential role in designing the major data pipeline. Specific scripts are executed to perform each stage, including *run_pipeline.py*, *run_spark_processing.py*, and *run_visualization.py*.

4.5 Technology Justification

Storage of data will be done using **HDFS** while **Apache Spark** will process the data efficiently. Data exploration will be done through **Jupyter Notebook** while **Ubuntu** will operate the entire system. Data cleaning will be conducted through the **pandas** library while **matplotlib** and **seaborn** libraries will generate graphs.

5. Methodology

5.1 Overview of the Three-Phase Pipeline

The system works in **three main steps (phases)**. Each step can run separately and produces output that is used in the next step. This makes the system easy to manage, test, and rerun without starting from the beginning.

5.2 Phase 1: Data Ingestion and Preparation

Firstly, the CSV file is loaded using pandas, and the data is summarized.

The following process cleans and transforms the dataset.

The transformed data will be stored in two file formats:

- CSV – for viewing purposes
- Parquet – for efficient reading within Apache Spark environment.

A summary of the data is also created which provides number of rows, column names, data type of columns, and number of deleted rows.

5.3 Phase 2: Distributed Processing with Apache Spark

Phase 2 uses the Spark pipeline module. First, all output directories are created before the start of the SparkSession since this ensures that Spark will not try to write to non-existing directories. The SparkSession operates in local multi-core mode with ERROR-level logging. Next, the cleaned Parquet dataset is loaded into a Spark DataFrame, followed by running disease analysis, demographic analysis, and anomaly detection in sequence through the execution of their respective modules. Each module writes its result in the form of Spark CSV part files to the appropriate output directories.

5.4 Phase 3: Visualisation

Phase 3 performs the loading of the Spark generated files via a helper function that combines the part files for each folder into one pandas DataFrame using Python's glob module.

This is followed by generation of seven charts such as counts of patients by condition, comparison of test results, distribution of diseases among genders, analysis of age groups, high number of days at hospital, high billing, and abnormal test results.

The last step involves saving the generated charts as PNG files in the *data/output/charts* folder.

6. Implementation

The project is organised as a Python repository with clear separation. The *src/* folder has two parts: *data_engineering* (pandas cleaning and transformation) and *spark_processing* (Spark analysis and visualisation). Scripts (*run_pipeline.py*, *run_spark_processing.py*, *run_visualization.py*) run each phase separately. Data is stored in *data/raw/*, *data/processed/*, and *data/output/*.

The data engineering layer uses pandas with three modules. *load_data* reads CSV files and shows their size. *clean_data* cleans the data using validation rules and keeps invalid values for checking. *transform_data* creates categories using *cut()* and extracts date features.

The Spark processing layer has five modules. *spark_session* creates the *SparkSession*. *load_processed_data* reads the Parquet file using *file://*. *disease_analysis* finds patient counts and test result distribution. *demographic_analysis* groups data by condition with gender and age. *anomaly_detection* filters patients with long stays, high billing, and abnormal results.

The project uses the Spark DataFrame API (not RDDs). It is easier to read, similar to SQL, and improves performance. Operations like *groupBy*, *filter*, and *select* run only when needed.

Results are saved using *write.csv()* with headers. Spark creates multiple part files. These are combined into pandas DataFrames using the *glob* module. This allows visualisation without needing Spark installed.

7. Results and Evaluation

7.1 Disease Prevalence Analysis

Table 4 presents the patient count for each of the eight medical conditions present in the dataset, derived from the top_medical_conditions Spark query.

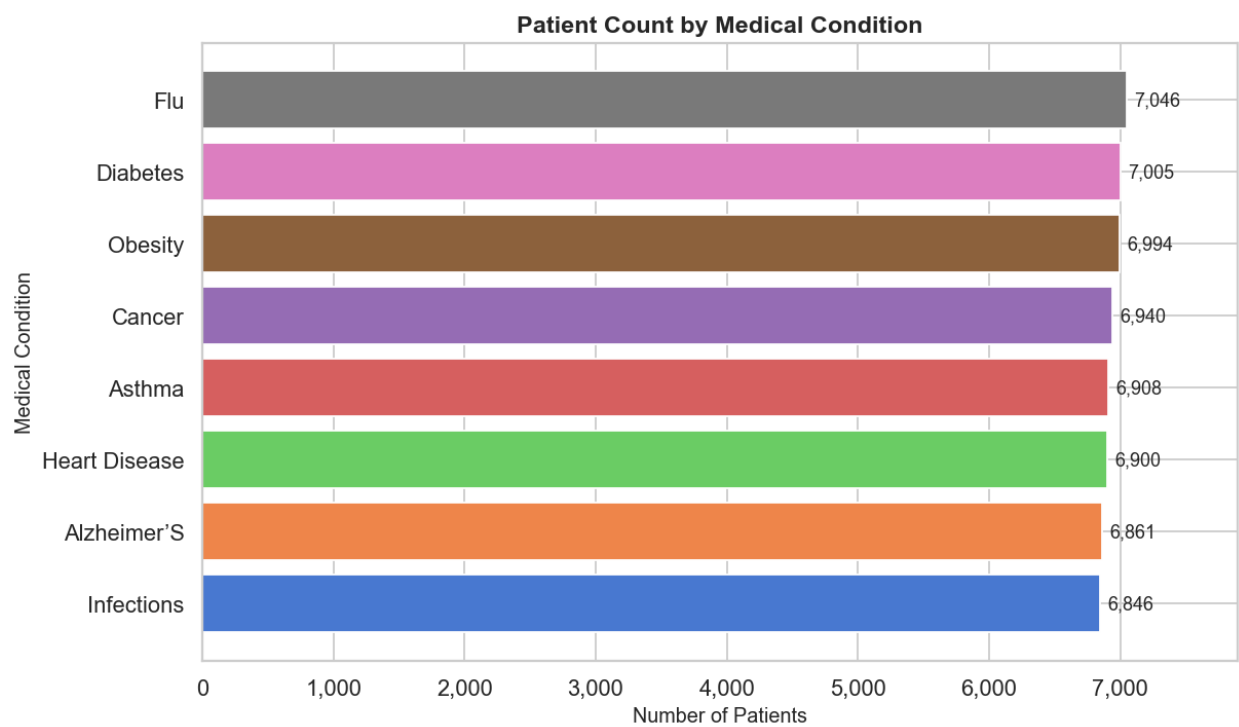


Figure 4: Patient Count by Medical Condition

The data distribution is almost even among the eight categories. There is very little variation in the number of patients ranging from 6,846 (Infections) to 7,046 (Flu), which represents less than a 3% variance. This indicates that the dataset has been artificially produced and balanced with an equal distribution of diseases. In reality, this cannot occur since the prevalence rates of diseases such as diabetes and hypertension are significantly higher compared to flu or infections. Therefore, the findings of this study need to be cautiously interpreted, considering that any differences observed are not related to the prevalence of the disease but the artificially constructed nature of the data.

7.2 Test Results Distribution

Table 5 presents the distribution of laboratory test results across the entire patient population.

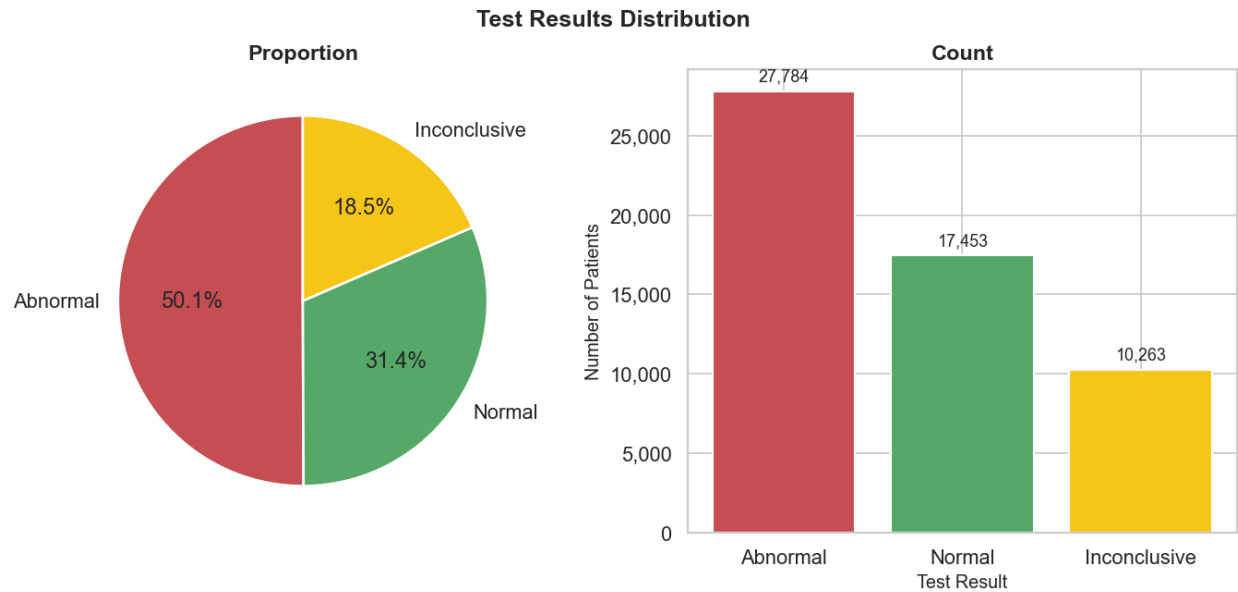


Figure 5: Test Results Distribution

Most of the patients (50.1%) show abnormal test results. This is not practical and constitutes the most significant issue within the data.

Typically, in actual health facilities, the percentage of patients with abnormal test results varies between 10%-20%, depending on the circumstances. Hence, a 50% ratio is quite unusual.

Some possible explanations for this anomaly include:

- A potential bias in the test_results variable, or
- The dataset treats all deviations from the standard range as abnormal, leading to higher numbers.

Therefore, this anomaly is deemed to be a data quality problem and needs to be verified with the data supplier before any medical decisions can be made.

7.3 Demographic Analysis

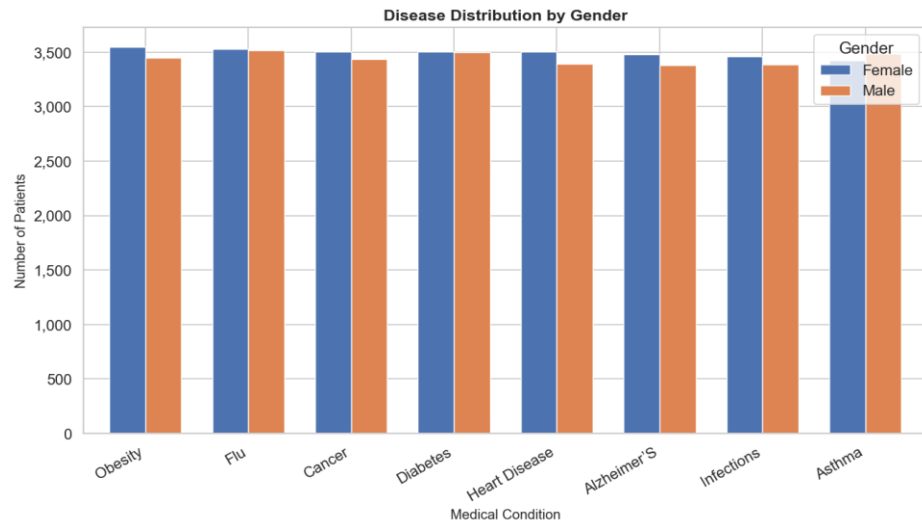


Figure 6: Disease Distribution by Gender

The figure shows almost perfect gender equality in all the eight diseases, whereby there are slightly more females than males in most cases by around 100 to 200 people. The lack of any disease with a significant gender bias is a further indication that the dataset used is artificially stratified since, one would expect more heart disease among men and more autoimmune diseases among women.

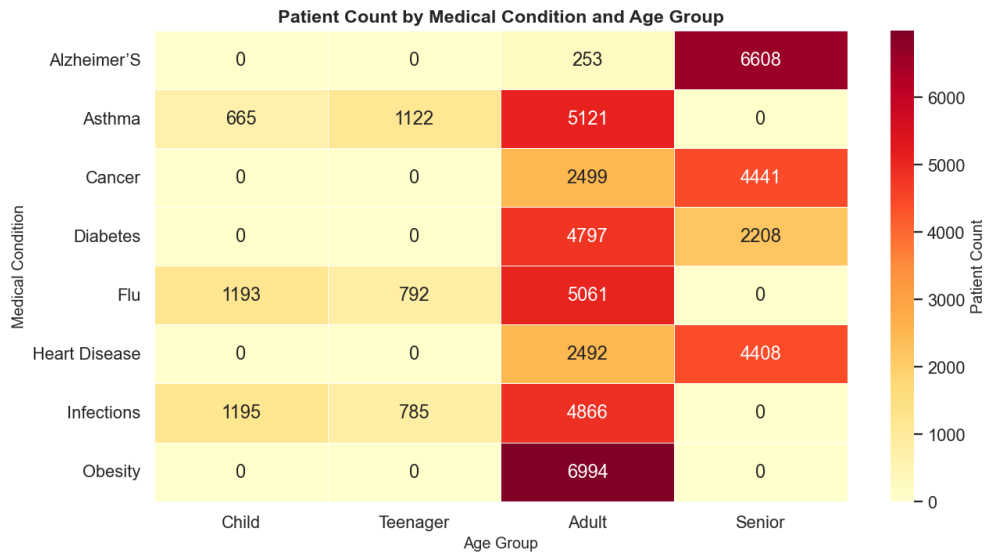


Figure 7: Patient Count by Medical Condition and Age Group

7.4 Anomaly Detection Results

Figure 8 summarises the three categories of anomalies detected by the Spark pipeline.

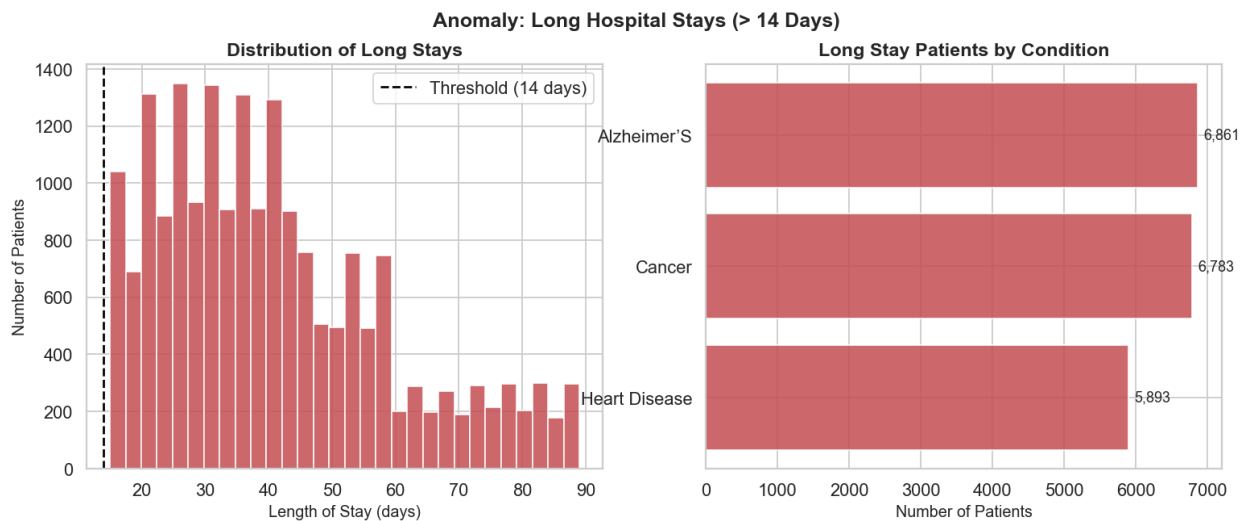


Figure 8: Anomaly Detection Summary

From the analysis of the data on long-term hospital stays, it was found that the disease most linked to long hospital stays was Alzheimer's Disease, with 6,861 patients, then Cancer, 6,783 patients, and heart disease, 5,893 patients. This conclusion was based on clinical logic since Alzheimer's patients need long-term treatment due to behavior management issues, cancer patients might need to undergo chemotherapy for longer periods, and heart disease patients might need to be observed after surgery.

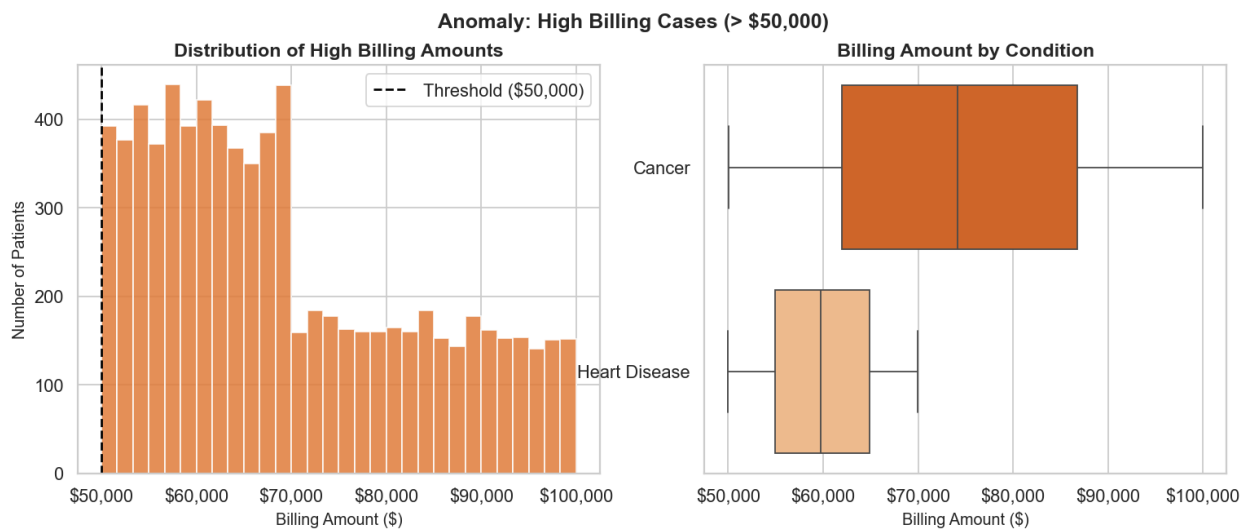


Figure 9: Anomaly: High Billing Cases

In the high billing analysis, Cancer was determined to have the highest median billing value more than \$50,000, with an IQR that stretched into the \$100,000 bracket. The distribution of high billing patients with heart disease had a smaller IQR, centered around the \$50,000-\$75,000 range. Such a difference may be expected since oncologic procedures are more expensive than cardiac procedures.

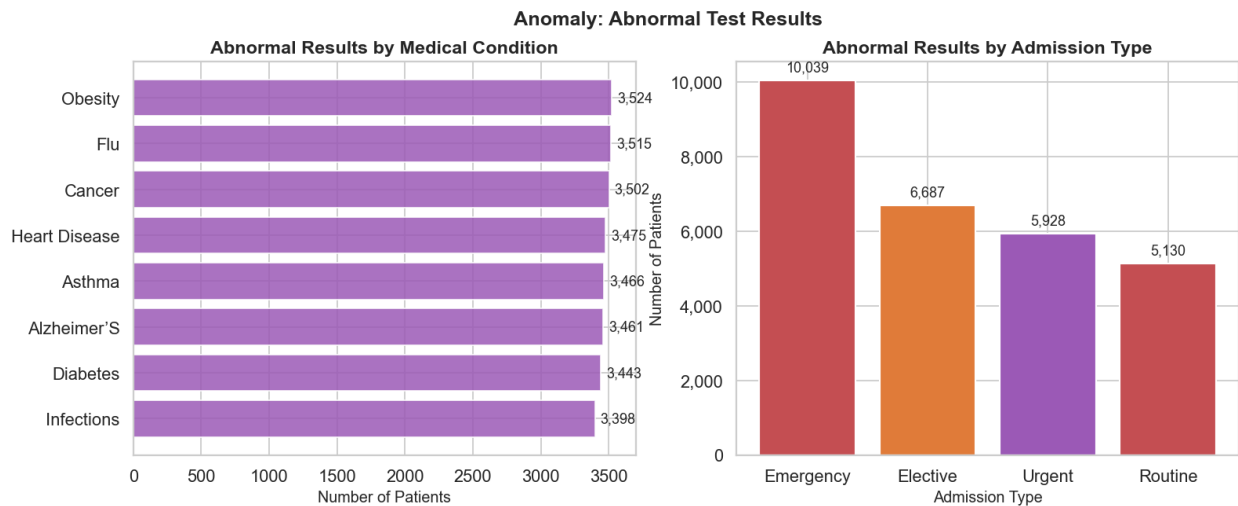


Figure 10: Anomaly: Abnormal Test Results

In respect of the abnormal tests based on mode of admission, there were the highest numbers of abnormal tests from the Emergency admissions at 10,039. This was followed by Elective admissions at 6,687, Urgent admissions at 5,928, and Routine admissions at 5,130. The expected trend would be in keeping with patient acuity status.

8. Discussion

8.1 Challenges Encountered

There were some issues when constructing and running this system.

First, there was a problem with file paths in Spark. Spark automatically attempts to use Hadoop's HDFS if Hadoop configuration is present. This resulted in local file paths being misdirected. We solved this issue by using `file://` before all local paths, which ensures Spark uses the local system.

Second, we had issues with saving files. Spark doesn't create the necessary folders when saving CSV files. This caused errors during execution. This issue was resolved by creating all folders using `os.makedirs()`.

Third, there was a data validation issue. Admission types were limited to three: Elective, Urgent and Emergency. But the data set also had another type of admission called "Routine." This has not been validated and was not checked, which may impact analysis.

Lastly, the age column was stored as a decimal and should not be. This was corrected by changing this to an integer so that the age does not have decimals.

8.2 Limitations

The biggest weakness of this study is the use of a synthetic dataset. Although the data is consistent, and contains some plausible age-condition relationships, the even distribution of diseases, the balanced gender distribution, and the unrealistically high percentage of abnormal test results (50.1%) suggest the data was created with statistical rather than clinical realism. Any inferences from this data about real healthcare analytics would thus be questionable until confirmed by real patient data.

Second, the use of Spark in local multi-core mode rather than a cluster is a limitation. Although local mode correctly illustrates the programming model of Spark and produces the same analytical results as a cluster, it lacks the network, serialisation and fault-tolerance mechanisms used in a distributed environment. Efficient horizontal scaling - adding nodes to a cluster and seeing a linear increase in throughput - is not possible in local mode.

The third problem is the anomaly detection thresholding method. Hard-coded rules (such as long stay = 14 days, high bill = \$50,000) are easy to program and understand but are not adaptive to the data and specific to the domain. A better alternative would be to use statistical techniques for anomaly detection, such as z-score thresholding, Isolation Forest or Local Outlier Factor, which take into account the distributional characteristics of the data.

8.3 Performance and Scalability

When running locally in parallel mode on a typical development environment, the Spark processing phase takes 30-90 seconds (depending on the number of available CPU cores and JVM warm-up). The data engineering phase takes less than 10 seconds on a 55,500-row dataset and involves all transformations in pandas on a single core. The visualisation phase (Phase 3) takes 15-30 seconds, depending on the type of plots to be produced.

In terms of scalability, a Parquet file format offers big data advantages in terms of larger datasets: this columnar format allows Spark to only read the columns necessary for a given query, and is compressed to reduce I/O. At 10 million records (a reasonable size for a regional hospital network over 5 years), the Spark processing phase would be expected to scale to completion within a similar time on a small (5-node) cluster, as Spark partitions data linearly. The data engineering phase would need to migrate from pandas (which can only use a single machine's memory) to Spark DataFrames for all the transformation steps to remain scalable.

9. Conclusion

This project has successfully built a full big data analytics pipeline for a large healthcare dataset using Apache Hadoop Distributed File System (HDFS), Spark, Jupyter Notebook and Ubuntu. The system is built in a transparent and modular fashion, including data cleaning, transformation, distributed processing, and visualisation. The pipeline was used to gain insights from the data. It highlighted the association of diseases with age, and detected abnormal test outcomes, prolonged hospital stays and costly cases. These findings show how big data technologies can help improve knowledge and decision making in healthcare. However, the project also identified some challenges. The high percentage of abnormal tests (50.1%) may indicate data quality problems, demonstrating the need for data validation in real systems. Overall, this project demonstrates the feasibility of big data technologies to process and analyse large healthcare data. In future, by adding real HDFS integration, anomaly detection with machine learning techniques, real-time data processing and interactive dashboards (using Apache Superset or Grafana), we can further improve the system and make it more effective in healthcare monitoring and analysis.

10. Workload Distribution

Name	Role	Contribution
W G M M Ajmal Mansoor (26775)	Spark & Hadoop Implementation	Worked with Apache Spark and Hadoop (HDFS), loaded data, ran basic operations like filtering and grouping, and supported data processing
K.B.A. Pasandul(30452)	Data Preprocessing & Cleaning	Cleaned the dataset, handled missing/invalid values, and prepared data for analysis
Chandrasinghe S S(29793)	Data Analysis & Visualisation	Identified anomalies and created charts and graphs for results
ABS Mahanama (27818)	System Design & Architecture	Designed the system structure and explained how data flows through the system
Pathirana R V (30486)	Evaluation & Results Analysis	Analysed outputs, explained results, and supported evaluation

11. References

- [1] P. Patil, *Healthcare Patient Admission Dataset*, Kaggle, 2024.
- [2] M. Armbrust, R. S. Xin and C. Lian, "Spark SQL: Relational data processing in Spark," in *Proceedings of the 2015 ACM SIGMOD International Conference*, 2015.
- [3] K. Shvachko, H. Kuang and S. Radia, "The Hadoop Distributed File System," in *IEEE 26th Symposium on Mass Storage Systems*, 2010.
- [4] T. White, *Hadoop: The Definitive Guide*, O'Reilly Media, Inc., 2012.
- [5] O. Hayırlı, "Anomaly detection in healthcare data using machine learning," *ResearchGate Publication*, 2025.